Lane2 Language Specification

Version: v1 draft
Status: working specification

Contents

1 Introduction

Lane2 is a strict, pure, expression-oriented functional programming language. Its first implementation target is a parser, a type checker, and an AST interpreter; later implementations may add bytecode compilation, a virtual machine, a linker, and effect handling.

Lane2 is intentionally small. It has no mutable state, assignment, trait system, method dispatch, module system, or implicit typeclass-style instance search in v1. Instead, v1 is centered on nominal data, first-class functions, bidirectional local type inference, exhaustive pattern matching, and explicit operation values opened into lexical scope.

This document specifies Lane2/Core v1: the platform-independent part of Lane2 that should behave the same across the initial AST interpreter and later execution backends.

1.1 Scope

Lane2/Core v1 covers:

- source structure and declarations;
- lexical scope and name resolution;
- nominal struct and enum types;
- function and block expressions;
- local type inference;
- pattern matching;
- `open`, `preopen`, and operation-based operators;
- unsafe builtin expressions.

The following are outside Lane2/Core v1:

- mutation and assignment;
- modules and imports;
- bytecode and virtual machine semantics;
- linker entrypoint selection;
- algebraic effects;
- type aliases;
- traits, typeclasses, and interfaces;
- tuples and collection syntax.

1.2 Compatibility

This specification is a v1 draft. No compatibility is promised between draft revisions. Language rules may be added, removed, or changed while the first implementation is being developed.

Once a v1 implementation is declared stable, this section should be replaced by an explicit compatibility policy.

1.3 Experimental Features

This draft does not distinguish stable and experimental language features. Every rule in this document is provisional until v1 is stabilized.

Future drafts may mark individual features as experimental when their surface syntax or semantics are intentionally unsettled.

1.4 Feedback

Design feedback is tracked in the Lane2 repository. Implementation changes should update this specification, `docs/language-v1.md`, `docs/prelude-v1.md`, `CONTEXT.md`, and ADRs when the change affects user-visible semantics.

1.5 Reference

When referencing this draft, use:

> Lane2 Language Specification : Lane2/Core v1 draft.

2 Syntax and Grammar

2.1 Notation

```
grammar        ::= { production }
production     ::= nonTerminal "::=" grammarExpression
grammarExpression ::=
    grammarSequence { "|" grammarSequence }
grammarSequence ::=
    { grammarItem }
grammarItem ::=
    terminal
  | nonTerminal
  | "[" grammarExpression "]"
  | "{" grammarExpression "}"
  | "(" grammarExpression ")"
  | grammarItem "?"
  | grammarItem "*"
  | grammarItem "+"

terminal       ::= "'" terminalText "'"
nonTerminal    ::= lowerCamelIdentifier

empty sequence denotes epsilon.
"A B" denotes sequencing.
"A | B" denotes choice.
"[ A ]" and "A?" denote optional occurrence.
"{ A }" and "A*" denote zero or more occurrences.
"A+" denotes one or more occurrences.
Parenthesized grammar expressions group without producing syntax.
```

2.2 Lexical Grammar

```
lexicalInput ::=
    (trivia | token)* eof

trivia ::=
    whitespace
  | lineTerminator
  | comment

token ::=
    keyword
  | reservedWord
  | identifier
  | intLiteral
  | stringLiteral
  | boolLiteral
  | operatorToken
  | punctuationToken

keyword ::=
    "struct"
  | "enum"
  | "fn"
  | "let"
  | "open"
  | "if"
  | "else"
```

```
  | "match"
  | "forall"
  | "builtin"

reservedWord ::=
    "effect"
  | "handler"
  | "module"
  | "import"
  | "pub"
  | "type"
  | "trait"
  | "interface"
  | "mut"
  | "return"

identifier ::=
    xidStart identifierContinue*
  | "_" identifierContinue+

identifierContinue ::=
    "_"
  | xidContinue
  | decimalDigit

boolLiteral ::=
    "true"
  | "false"

intLiteral ::=
    decimalDigit+

stringLiteral ::=
    "\"" stringElement* "\""

stringElement ::=
    asciiStringCharacter
  | stringEscape

asciiStringCharacter ::=
    asciiCodePointExceptControlBackslashQuote

stringEscape ::=
    "\\" "\\"
  | "\\" "\""
  | "\\" "n"
  | "\\" "r"
  | "\\" "t"
  | "\\" "x" asciiHexByte

asciiHexByte ::=
    asciiHexLowByte
  | asciiHexHighByte

asciiHexLowByte ::=
    ("0" | "1" | "2" | "3" | "4" | "5" | "6") asciiHexDigit

asciiHexHighByte ::=
    "7" ("0" | "1" | "2" | "3" | "4" | "5" | "6" | "7")

operatorToken ::=
    "+"
  | "-"
```

```
  | "*"
  | "/"
  | "%"
  | "=="
  | "!="
  | "<"
  | "<="
  | ">"
  | ">="
  | "&&"
  | "||"
  | "!"
  | "|>"

punctuationToken ::=
    "("
  | ")"
  | "{"
  | "}"
  | "["
  | "]"
  | ","
  | "."
  | ":"
  | "::"
  | "->"
  | "=>"
  | "="
  | "_"

whitespace ::=
    U+0009
  | U+000B
  | U+000C
  | U+0020

lineTerminator ::=
    U+000A
  | U+000D
  | U+000D U+000A

decimalDigit ::=
    "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

asciiHexDigit ::=
    decimalDigit
  | "a" | "b" | "c" | "d" | "e" | "f"
  | "A" | "B" | "C" | "D" | "E" | "F"

xidStart ::=
    any Unicode code point with the XID_Start property

xidContinue ::=
    any Unicode code point with the XID_Continue property
```

Lexical analysis uses maximal munch. If more than one token class matches the same maximal source span, keyword, reservedWord, and boolLiteral take priority over identifier.

2.3 Syntax Grammar

```
sourceFile ::=
    topLevelDeclaration* eof

topLevelDeclaration ::=
```

```
    structDeclaration
  | enumDeclaration
  | functionDeclaration
  | topLevelLetDeclaration
  | anonymousTopLevelLetDeclaration
  | openDeclaration

structDeclaration ::=
    "struct" typeName typeParameters? "{" structMember* "}"

structMember ::=
    fieldDeclaration
  | fieldForwardingDeclaration

fieldDeclaration ::=
    fieldName space ":" space type

fieldForwardingDeclaration ::=
    "open" valueName

enumDeclaration ::=
    "enum" typeName typeParameters? "{" enumVariant* "}"

enumVariant ::=
    variantName enumPayload?

enumPayload ::=
    "(" commaSeparatedTypes? ")"

functionDeclaration ::=
    "fn" typeParameters? functionName "(" commaSeparatedParameters? ")" "->" type block

parameter ::=
    valueName space ":" space type

topLevelLetDeclaration ::=
    "let" valueName space ":" space type "=" expression

anonymousTopLevelLetDeclaration ::=
    "let" space ":" space type "=" expression

localLetDeclaration ::=
    "let" valueName typeAnnotation? "=" expression

typeAnnotation ::=
    space ":" space type

openDeclaration ::=
    "open" valueName

type ::=
    forallType
  | typeConstructor typeArguments?
  | functionType

typeConstructor ::=
    typeName

typeArguments ::=
    "[" commaSeparatedTypes "]"

typeParameters ::=
    "[" commaSeparatedTypeParameters "]"
```

```
forallType ::=
    "forall" commaSeparatedTypeParameters "." functionType

functionType ::=
    "(" commaSeparatedTypes? ")" "->" type

expression ::=
    ifExpression
  | matchExpression
  | functionLiteral
  | block
  | pipelineExpression

pipelineExpression ::=
    binaryExpression { "|>" pipelineRhs }

pipelineRhs ::=
    callExpression
  | functionLiteral

binaryExpression ::=
    unaryExpression { binaryOperator unaryExpression }

unaryExpression ::=
    unaryOperator unaryExpression
  | callExpression

callExpression ::=
    fieldExpression { callSuffix }

callSuffix ::=
    "(" commaSeparatedExpressions? ")"

fieldExpression ::=
    primaryExpression { "." fieldName }

primaryExpression ::=
    literal
  | valueName
  | qualifiedVariantExpression
  | structLiteral
  | builtinExpression
  | "(" expression ")"
  | "(" ")"

literal ::=
    intLiteral
  | stringLiteral
  | boolLiteral

qualifiedVariantExpression ::=
    typeName typeArguments? "::" variantName variantArguments?

variantArguments ::=
    "(" commaSeparatedExpressions? ")"

structLiteral ::=
    typeName typeArguments? "::" "{" commaSeparatedStructLiteralFields "}"

structLiteralField ::=
    fieldName
  | fieldName ":" expression
```

```
builtinExpression ::=
    "builtin" "(" stringLiteral ")"

functionLiteral ::=
    "fn" typeParameters? "(" commaSeparatedFunctionLiteralParameters? ")"
functionReturnAnnotation? block

functionLiteralParameter ::=
    valueName
  | valueName space ":" space type

functionReturnAnnotation ::=
    "->" type

block ::=
    "{" localItem* expression "}"

localItem ::=
    localLetDeclaration
  | functionDeclaration
  | openDeclaration

ifExpression ::=
    "if" expression block "else" block

matchExpression ::=
    "match" expression "{" matchArm* "}"

matchArm ::=
    pattern "=>" expression

pattern ::=
    "_"
  | valueName
  | literal
  | qualifiedVariantPattern
  | structPattern

qualifiedVariantPattern ::=
    typeName "::" variantName patternArguments?

patternArguments ::=
    "(" commaSeparatedPatterns? ")"

structPattern ::=
    typeName "::" "{" commaSeparatedStructPatternFields "}"

structPatternField ::=
    fieldName
  | fieldName ":" pattern

binaryOperator ::=
    "+" | "-" | "*" | "/" | "%"
  | "==" | "!=" | "<" | "<=" | ">" | ">="
  | "&&" | "||"

unaryOperator ::=
    "-" | "!"

commaSeparatedTypes ::=
    type ("," type)* ","?
```

```
commaSeparatedTypeParameters ::=
    typeParameter ("," typeParameter)* ","?

commaSeparatedParameters ::=
    parameter ("," parameter)* ","?

commaSeparatedExpressions ::=
    expression ("," expression)* ","?

commaSeparatedFunctionLiteralParameters ::=
    functionLiteralParameter ("," functionLiteralParameter)* ","?

commaSeparatedStructLiteralFields ::=
    structLiteralField ("," structLiteralField)* ","?

commaSeparatedPatterns ::=
    pattern ("," pattern)* ","?

commaSeparatedStructPatternFields ::=
    structPatternField ("," structPatternField)* ","?

typeName ::=
    identifier

functionName ::=
    identifier

valueName ::=
    identifier

fieldName ::=
    identifier

variantName ::=
    identifier

typeParameter ::=
    identifier

space ::=
    whitespace+
```

The precedence and associativity of `binaryOperator`, `unaryOperator`, calls, field access, and pipeline expressions are defined in “Operators”.

2.4 Comments

```
comment ::=
    lineComment
  | blockComment

lineComment ::=
    "/" "/" lineCommentCharacter* lineTerminator?

lineCommentCharacter ::=
    any Unicode code point except U+000A or U+000D

blockComment ::=
    "/" "*" blockCommentCharacter* "*" "/"

blockCommentCharacter ::=
    any Unicode code point sequence that does not start "*/"
```

Comments are trivia. Comments do not appear in the syntactic grammar.

3 Type System

3.1 Notations

Notation	Meaning
T, U, R	types
A	type variable
C	type constructor
S	struct type constructor
E	enum type constructor
e, c, f, a	expressions
x	value binder
n, m	natural numbers
Δ	type-constructor context
Θ	type-variable context
Γ	value typing context
$\Delta\Theta \vdash T \quad \text{type}$	well-formed type
$C[T_1, ..., T_n]$	nominal type application
$\forall A_1, ..., A_n.T$	universal type
$T \equiv U$	type equality
$\Gamma \vdash e : T$	expression typing
$\dfrac{P}{Q}$	rule with premise P and conclusion Q
name	abstract syntax constructor

Table 1: Type system notations

3.2 Primitive Types

Lane2/Core v1 has four primitive types: `Unit`, `Bool`, `Int`, and `String`.

Primitive formation. Each primitive type is well-formed in every type environment.

$$\Delta\Theta \vdash \text{Unit} \quad \text{type}$$

$$\Delta\Theta \vdash \text{Bool} \quad \text{type}$$

$$\Delta\Theta \vdash \text{Int} \quad \text{type}$$

$$\Delta\Theta \vdash \text{String} \quad \text{type}$$

Unit introduction. The expression `()` introduces a `Unit` value.

`()`

$$\Gamma \vdash () : \text{Unit}$$

Unit elimination. Lane2/Core v1 has no dedicated elimination form for `Unit`.

Bool introduction. Boolean literals introduce `Bool` values.

```
true
false
```

$$\Gamma \vdash \text{true} : \text{Bool}$$

$$\Gamma \vdash \text{false} : \text{Bool}$$

Bool elimination. `if` eliminates a `Bool` value.

```
if condition {
  then_value
} else {
  else_value
}
```

$$\frac{\Gamma \vdash c : \text{Bool} \quad \Gamma \vdash e_1 : T \quad \Gamma \vdash e_2 : T}{\Gamma \vdash \text{if}(c, e_1, e_2) : T}$$

Int introduction. Integer literals introduce `Int` values.

```
42
```

$$\Gamma \vdash n : \text{Int}$$

Int elimination. Lane2/Core v1 has no built-in syntactic eliminator for `Int`. Integer operations are typed through required intrinsics and prelude operation values.

String introduction. ASCII string literals introduce `String` values.

```
"lane"
```

$$\Gamma \vdash s : \text{String}$$

String elimination. Lane2/Core v1 has no built-in syntactic eliminator for `String`. String equality is typed through the prelude.

Primitive type equality. Primitive types are equal only to themselves.

$$\text{Unit} \equiv \text{Unit}$$

$$\text{Bool} \equiv \text{Bool}$$

$$\text{Int} \equiv \text{Int}$$

$$\text{String} \equiv \text{String}$$

3.3 Function Types

Function formation.

$$\frac{\Delta\Theta \vdash T_1 \quad \text{type} \quad ... \quad \Delta\Theta \vdash T_n \quad \text{type} \quad \Delta\Theta \vdash R \quad \text{type}}{\Delta\Theta \vdash (T_1, ..., T_n) \to R \quad \text{type}}$$

Function introduction.

```
fn(x : Int, y : Int) -> Int {
  x + y
}
```

$$\frac{\Gamma, x_1 : T_1, ..., x_n : T_n \vdash e : R}{\Gamma \vdash \text{fn}((x_1 : T_1), ..., (x_n : T_n), R, e) : (T_1, ..., T_n) \to R}$$

Function elimination.

```
f(a, b)
```

$$\frac{\Gamma \vdash f : (T_1, ..., T_n) \to R \quad \Gamma \vdash a_1 : T_1 \quad ... \quad \Gamma \vdash a_n : T_n}{\Gamma \vdash f(a_1, ..., a_n) : R}$$

Function type equality.

$$\frac{T_1 \equiv U_1 \quad ... \quad T_n \equiv U_n \quad R \equiv S}{(T_1, ..., T_n) \to R \equiv (U_1, ..., U_n) \to S}$$

Function types are uncurried. `(T1, T2) -> R` is not the same type object as `(T1) -> (T2) -> R`.

3.4 Generic Function Types

Generic function formation.

```
forall A. (A) -> A
```

$$\frac{\Delta\Theta, A_1, ..., A_n \vdash (T_1, ..., T_m) \to R \ \ \text{type}}{\Delta\Theta \vdash \forall A_1, ..., A_n.(T_1, ..., T_m) \to R \ \ \text{type}}$$

Generic function introduction.

```
fn[A](value : A) -> A {
  value
}
```

A generic function literal or named generic function introduces a generic function value.

Generic function elimination.

Generic function calls instantiate type parameters at the use site when the use site is unambiguous.

```
id(1)
```

Generic function type equality.

$$\frac{(T_1, ..., T_m) \to R \equiv (U_1, ..., U_m) \to S}{\forall A_1, ..., A_n.(T_1, ..., T_m) \to R \equiv \forall A_1, ..., A_n.(U_1, ..., U_m) \to S}$$

Generic function type equality compares the number of type parameters and the function types under corresponding bound type parameters.

3.5 Struct Types

Struct formation.

```
struct Point {
  x : Int
  y : Int
}
```

A `struct` declaration introduces a nominal type constructor. If the declaration has type parameters, the type constructor arity is the number of declared type parameters.

$$\frac{\text{S has arity} \ \ n \ \ \text{in} \ \ \Delta \quad \Delta\Theta \vdash T_1 \ \ \text{type} \quad ... \quad \Delta\Theta \vdash T_n \ \ \text{type}}{\Delta\Theta \vdash \text{S}[T_1, ..., T_n] \ \ \text{type}}$$

Struct introduction.

```
Point::{ x: 1, y: 2 }
```

A qualified struct literal introduces a struct value. It must provide every field exactly once.

Struct elimination.

```
p.x
match p {
  Point::{ x, y } => x + y
```

```
}
open point_ops
```

Struct values are eliminated by field access, struct patterns, and `open`.

Struct type equality.

$$\frac{\mathrm{S}\ \text{ is the same nominal type constructor as }\ \mathrm{S} \quad T_1 \equiv U_1 \quad \dots \quad T_n \equiv U_n}{\mathrm{S}[T_1, \dots, T_n] \equiv \mathrm{S}[U_1, \dots, U_n]}$$

Distinct struct type constructors are not equal, even when their fields have the same names and types.

3.6 Enum Types

Enum formation.

```
enum Option[A] {
  none
  some(A)
}
```

An `enum` declaration introduces a nominal type constructor. If the declaration has type parameters, the type constructor arity is the number of declared type parameters.

$$\frac{\mathrm{E}\ \text{has arity}\ \ n\ \ \text{in}\ \ \Delta \quad \Delta\Theta \vdash T_1\ \ \text{type} \quad \dots \quad \Delta\Theta \vdash T_n\ \ \text{type}}{\Delta\Theta \vdash \mathrm{E}[T_1, \dots, T_n]\ \ \text{type}}$$

Enum introduction.

```
Option::none
Option::some(1)
some(1)
```

A qualified enum variant expression introduces an enum value. An unqualified enum variant expression is allowed only when the variant name resolves without ambiguity.

Enum elimination.

```
match value {
  Option::none => fallback
  Option::some(x) => x
}
```

Enum values are eliminated by exhaustive `match`.

Enum type equality.

$$\frac{\mathrm{E}\ \text{ is the same nominal type constructor as }\ \mathrm{E} \quad T_1 \equiv U_1 \quad \dots \quad T_n \equiv U_n}{\mathrm{E}[T_1, \dots, T_n] \equiv \mathrm{E}[U_1, \dots, U_n]}$$

Distinct enum type constructors are not equal, even when their variants have the same names and payload types.

3.7 Local Type Inference

Lane2 permits local type inference only at syntactic positions where the omitted type is determined locally.

A binding's type is not determined from later uses of the bound name.

Local `let` bindings may omit type annotations only when the initializer has a type without using later references to the bound name.

Function literals may omit parameter types only when the immediately surrounding context provides a function type. Otherwise, every value parameter of a function literal must have an explicit type annotation.

Generic function literals without an immediately surrounding generic function type must explicitly declare type parameters and explicitly annotate value parameters.

Generic function calls and generic data constructors may instantiate type parameters at the use site when the use site is unambiguous.

3.8 Static Semantics Summary

$$\Gamma \vdash \text{sourceFile} \quad \text{ok}$$

means the source file is syntactically valid, all referenced types are well-formed, all names resolve without ambiguity, every expression has a type, and all match expressions are exhaustive.

A Lane2/Core v1 source file is statically valid only if:

1. every type annotation denotes a well-formed type;
2. every expression has the type required by its enclosing construct;
3. every local binding obeys sequential local scope;
4. every top-level value and top-level open obeys ordered value scope;
5. every function body checks under the source file's top-level environment;
6. every match expression is exhaustive;
7. every unresolved name, ambiguous name, ill-formed type, or type mismatch is rejected.

4 Builtins

`builtin("...")` is an unsafe expression.

The checker does not interpret intrinsic strings. Instead, `builtin` receives its type from direct expected context:

```
fn int_add(a : Int, b : Int) -> Int {
  builtin("%i64_add")
}
```

This is valid because the function return type provides the expected type `Int` for the body expression.

This is invalid:

```
let x = builtin("%anything")
```

There is no direct expected type.

Incorrect builtin use can produce undefined behavior. Lane2′s type safety guarantee applies only to programs that do not misuse `builtin`.

4.1 Required Intrinsics

A conforming Lane2/Core v1 implementation provides these portable intrinsic names:

Intrinsic	Expected type
`%i64_add`	`(Int, Int) -> Int`
`%i64_sub`	`(Int, Int) -> Int`
`%i64_mul`	`(Int, Int) -> Int`
`%i64_div`	`(Int, Int) -> Int`
`%i64_rem`	`(Int, Int) -> Int`
`%i64_neg`	`(Int) -> Int`
`%i64_equal`	`(Int, Int) -> Bool`
`%i64_less`	`(Int, Int) -> Bool`
`%string_equal`	`(String, String) -> Bool`

Table 2: Required intrinsic names and expected types

Other intrinsic names are implementation-defined unsafe builtins.

`%bool_and`, `%bool_or`, `%bool_not`, and `%bool_equal` are not required intrinsics. The standard prelude defines boolean operations as ordinary Lane2 functions and anonymous operation values using `if`; && and `||` supply their right operands as thunks.

5 Prelude

The standard prelude is implementation-supplied Lane2 source checked before user code. It is not a module system.

Prelude declarations provide standard operation structs, primitive wrappers around required intrinsics, derived primitive operations written in Lane2, and anonymous top-level values that populate the initial preopen namespace.

Prelude entries are ordinary Lane2 values and types except where the compiler recognizes operation field pairs for operator aliases.

5.1 Standard Operation Structs

Arithmetic operations:

```
struct Add[T] {
  add : (T, T) -> T
}

struct Sub[T] {
  sub : (T, T) -> T
}

struct Mul[T] {
  mul : (T, T) -> T
}

struct Div[T] {
  div : (T, T) -> T
}

struct Rem[T] {
  rem : (T, T) -> T
}

struct Neg[T] {
  neg : (T) -> T
}
```

Boolean operations:

```
struct And {
  and : (Bool, () -> Bool) -> Bool
}

struct Or {
  or : (Bool, () -> Bool) -> Bool
}

struct Not {
  not : (Bool) -> Bool
}
```

Equality and ordering operations:

```
struct Equal[T] {
  equal : (T, T) -> Bool
  not_equal : (T, T) -> Bool
}

struct Compare[T] {
  equal_impl : Equal[T]
  open equal_impl
  less : (T, T) -> Bool
  less_eq : (T, T) -> Bool
```

```
  greater : (T, T) -> Bool
  greater_eq : (T, T) -> Bool
}
```

Operation laws are API conventions, not compiler-checked rules.

5.2 Standard Preopen Values

The prelude may populate the initial preopen namespace with anonymous top-level values. For example, primitive arithmetic and boolean operations can be exposed as follows:

```
let : Add[Int] = Add::{ add: int_add }
let : Sub[Int] = Sub::{ sub: int_sub }
let : Mul[Int] = Mul::{ mul: int_mul }
let : Div[Int] = Div::{ div: int_div }
let : Rem[Int] = Rem::{ rem: int_rem }
let : Neg[Int] = Neg::{ neg: int_neg }

let int_equal_ops : Equal[Int] = make_equal(int_equal)
let : Compare[Int] = make_compare(int_equal_ops, int_less)

let : And = And::{ and: bool_and }
let : Or = Or::{ or: bool_or }
let : Not = Not::{ not: bool_not }
let : Equal[Bool] = Equal::{
  equal: fn(a : Bool, b : Bool) {
    if a {
      b
    } else {
      bool_not(b)
    }
  },
  not_equal: fn(a : Bool, b : Bool) {
    if a {
      bool_not(b)
    } else {
      b
    }
  },
}

let : Equal[String] = make_equal(string_equal)
```

Boolean conjunction, disjunction, negation, and equality do not require boolean intrinsics; they can be defined with ordinary `if` expressions in the prelude.

6 Declarations

Declarations introduce types, functions, values, and open scope extensions.

6.1 Top-Level Declarations

Top-level declarations are described by the `topLevelDefinition` grammar in “Syntax and Grammar”.

Top-level forms include struct declarations, enum declarations, named function declarations, typed value declarations, anonymous typed value declarations, and open declarations.

6.2 Struct Declarations

Struct declarations use the grammar in “Structs, Enums, and Open”.

6.3 Enum Declarations

Enum declarations use the grammar in “Structs, Enums, and Open”.

6.4 Function Declarations

Functions are uncurried. There is no automatic currying or partial application.

Function definitions use block bodies:

```
functionDeclaration:
    'fn' [ typeParameters ] functionName '(' [ parameter { ',' parameter } [ ',' ] ] ')' '->'
type block

parameter:
    valueName ':' type

functionLiteral:
    'fn' [ typeParameters ] '(' [ functionLiteralParameter { ',' functionLiteralParameter }
[ ',' ] ] ')' [ '->' type ] block

functionLiteralParameter:
    valueName
  | valueName ':' type
```

```
fn add(a : Int, b : Int) -> Int {
  a + b
}
```

There is no `= expr` function body form.

Function result types use `->`, not `:`.

All named functions must state every parameter type and the result type.

Generic named functions place type parameters after `fn` and before the name:

```
fn[A] id(value : A) -> A {
  value
}
```

Function parameters are positional in v1. Labeled function parameters are not supported.

Function literals produce function values:

```
let f : (Int, Int) -> Int = fn(a, b) {
  a + b
}
```

Generic function literals place type parameters after `fn`:

```
let id = fn[A](value : A) {
  value
}
```

6.5 Let Declarations

Named top-level `let` declarations must include type annotations. Anonymous top-level `let` declarations must include type annotations and must have a struct type.

Local `let` declarations may omit type annotations when their initializer can synthesize a type by local type inference.

7 Scopes and Bindings

Top-level type and function definitions form recursive definition groups.

Top-level `struct`, `enum`, and `fn` definitions may refer to each other regardless of textual order.

Top-level value definitions and top-level `open` declarations follow ordered value scope:

- a top-level `let` initializer may refer only to values already available at that declaration point;
- a top-level `open` may open only a value already available at that declaration point;
- a top-level function body may refer to any top-level value or function, even one declared later.

Example:

```
fn read_x() -> Int {
  x
}

let x : Int = 1
```

This is valid because the function body sees the complete top-level environment.

```
let y : Int = x
let x : Int = 1
```

This is invalid because top-level value initializers follow ordered value scope.

7.1 Namespaces

Lane2 has separate type and value namespaces.

Type and value names may use the same spelling:

```
enum Option[A] {
  none
  some(A)
}

let Option : Int = 42
```

Syntax of the form `Type::member` resolves `Type` in the type namespace.

```
let x : Option[Int] = Option::some(1)
```

The `Option` on the left of `::` denotes the enum type, not the value named `Option`.

7.2 Local Bindings

Local `let` bindings are sequential. A local binding is visible only to later items and the final expression in the same block.

Local `let` bindings may omit type annotations when their initializer can synthesize a type by local type inference:

```
let x = 1
```

Local named functions are also sequential. They may call themselves, but local mutually recursive groups and forward references are not supported:

```
fn f(n : Int) -> Int {
  fn loop(x : Int) -> Int {
    if x == 0 {
      0
    } else {
      loop(x - 1)
    }
  }
  loop(n)
}
```

Local value names may shadow earlier local value names.

7.3 Open Scope Extensions

`open value` opens a struct value.

The operand of `open` must be a visible value name with a struct type. It cannot be an arbitrary expression:

```
open ops
```

This is invalid:

```
open make_ops(10)
```

An open scope extension exposes the struct field values as unqualified names from the declaration point to the end of the current lexical scope.

At top level, `open` extends to the end of the file:

```
open int_add_ops

fn add_one(x : Int) -> Int {
  x + 1
}
```

Inside blocks, `open` is a local item and must appear before the final expression:

```
{
  open int_add_ops
  x + y
}
```

Local resolution uses the nearest preceding local binding or open scope extension, then falls back to preopen.

7.4 Preopen

The preopen namespace is a default-open namespace populated by anonymous top-level values.

```
let : Add[Int] = Add::{ add: int_add }
```

An anonymous top-level value must have a struct type. It exposes field values, not generated accessors.

Prelude-provided anonymous values are checked before user code, so their preopen entries are visible throughout user code.

User-defined anonymous top-level values extend preopen from their declaration point to the end of the top-level scope.

Preopen conflicts are errors.

Top-level `open` belongs to the global layer and conflicts with ordinary top-level names or preopened names. Local `open` may shadow preopen inside its lexical scope.

8 Expressions

Expressions compute values. Lane2 v1 is expression-oriented and has no expression statements.

8.1 Blocks

A block expression contains local items followed by one final expression.

```
block:
    '{' { localItem } expression '}'

localItem:
    localLet
  | localFunctionDeclaration
  | openDeclaration

localLet:
    'let' valueName [ ':' type ] '=' expression

topLevelLet:
    'let' valueName ':' type '=' expression

anonymousTopLevelLet:
    'let' ':' type '=' expression

openDeclaration:
    'open' valueName
```

```
{
  let x = 1
```

```
  fn double(n : Int) -> Int {
    n + n
  }
  double(x)
}
```

Allowed local items:

- `let`,
- named `fn`,
- `open`.

Local type definitions are not supported in v1.

A block does not contain expression statements. This is invalid:

```
{
  f(x)
  g(y)
}
```

An empty block is invalid. Write `()` explicitly when a `Unit` value is required.

8.2 Conditional Expressions

`if` is an expression:

```
if condition {
  then_value
} else {
  else_value
}
```

The `else` branch is mandatory.

Both branches must have the same type.

8.3 Calls, Field Access, and Pipeline

Function call:

```
f(x, y)
```

Field access:

```
ops.add
```

Field access followed by call:

```
ops.add(x, y)
```

This is not method call syntax. Lane2 v1 has no method receiver lookup.

Pipeline syntax is supported:

```
value |> f(a, b)
```

It rewrites to:

```
f(value, a, b)
```

The right-hand side of `|>` must be a call or function literal.

The following are invalid:

```
value |> f
value |> f(_, y)
```

Placeholders are not supported.

9 Structs, Enums, and Open

```
structDeclaration:
    'struct' typeName [ typeParameters ] '{' { structMember } '}'

structMember:
    fieldDeclaration
  | fieldForwarding

fieldDeclaration:
    fieldName ':' type

fieldForwarding:
    'open' fieldName

structLiteral:
    typeName [ typeArguments ] '::' '{' structLiteralField { ',' structLiteralField } [ ',' ]
'}'

structLiteralField:
    fieldName
  | fieldName ':' expression
```

Struct literals are qualified:

```
let p : Point = Point::{ x: 1, y: 2 }
```

Struct field punning is allowed:

```
let p : Point = Point::{ x, y }
```

This is equivalent to:

```
let p : Point = Point::{ x: x, y: y }
```

Struct literals must provide every field exactly once.

The following are not supported:

- anonymous record literals,
- default fields,
- spread,
- copy update,
- field update.

Fields are accessed with dot syntax:

```
p.x
```

V1 has no field visibility modifiers. Every struct field is accessible wherever the struct value is accessible.

9.1 Enums

```
enumDeclaration:
    'enum' typeName [ typeParameters ] '{' { enumVariant } '}'

enumVariant:
    variantName [ '(' [ type { ',' type } [ ',' ] ] ')' ]

qualifiedVariantExpression:
    typeName [ typeArguments ] '::' variantName [ '(' [ expression { ',' expression } [ ',' ] ]
')' ]

unqualifiedVariantExpression:
    variantName [ '(' [ expression { ',' expression } [ ',' ] ] ')' ]
```

Enum variants are scoped to their enum.

Variant names may be lowercase or uppercase. Capitalization has no semantic role.

Payloadless variants are values and are written without call parentheses:

```
let x : Option[Int] = Option::none
```

This is invalid:

```
Option::none()
```

Variant payloads are positional:

```
enum Expr {
  int(Int)
  add(Expr, Expr)
}
```

Labeled variant payloads are not part of v1. Named product data must be represented with a struct:

```
struct Node[A] {
  left : Tree[A]
  value : A
  right : Tree[A]
}

enum Tree[A] {
  leaf(A)
  node(Node[A])
}
```

In expressions, unqualified variants are allowed when unambiguous:

```
let x : Option[Int] = some(1)
```

If more than one visible enum has a variant named `some`, the unqualified expression is invalid unless another directly local rule disambiguates it. A qualified variant is always allowed:

```
let x : Option[Int] = Option::some(1)
```

In patterns, variants must always be qualified.

9.2 Openable Struct Values

Only struct values can be opened.

Opening a struct value exposes its field values as unqualified bindings according to the scope rules in "Scopes and Bindings".

Enum values, function values, primitive values, and arbitrary expressions are not openable.

9.3 Struct Field Forwarding

A struct declaration may contain `open field` entries:

```
struct Compare[T] {
  equal_impl : Equal[T]
  open equal_impl
  less : (T, T) -> Bool
  less_eq : (T, T) -> Bool
  greater : (T, T) -> Bool
  greater_eq : (T, T) -> Bool
}
```

When a value of this struct type is opened, forwarded fields are exposed too.

Struct field forwarding affects only what is exposed by opening the containing struct value. It does not open the field inside ordinary function bodies.

10 Pattern Matching

`match` is an expression:

```
matchExpression:
    'match' expression '{' { matchArm } '}'

matchArm:
    pattern '=>' expression

pattern:
    '_'
  | valueName
  | literal
  | qualifiedVariantPattern
  | structPattern

qualifiedVariantPattern:
    typeName '::' variantName [ '(' [ pattern { ',' pattern } [ ',' ] ] ')' ]

structPattern:
    typeName '::' '{' structPatternField { ',' structPatternField } [ ',' ] '}'

structPatternField:
    fieldName
  | fieldName ':' pattern
```

```
match value {
  Option::none => fallback
  Option::some(x) => x
}
```

Match arms use `pattern => expression`.

Every match must be exhaustive.

V1 patterns:

- wildcard pattern: _,
- variable binding pattern: `name`,
- literal pattern,
- qualified enum variant pattern,
- qualified struct pattern.

Enum variants in patterns must be qualified:

```
Option::some(x)
Option::none
```

Bare identifiers in patterns are always variable bindings:

```
match value {
  x => x
}
```

Struct patterns use qualified syntax and must list all fields:

```
match p {
  Point::{ x, y } => x + y
}
```

Struct pattern punning is allowed. Explicit renaming is allowed:

```
match p {
  Point::{ x: a, y: b } => a + b
}
```

Rest patterns, spread patterns, guards, or-patterns, as-patterns, and `is` pattern expressions are not supported in v1.

11 Operators

Operators are aliases for recognized prelude operation fields.

There is no type-directed instance search. An operator is available only when the relevant operation field is available through preopen or an open scope extension.

Primitive operators are not special-cased. For example, `1 + 2` requires an available `Add::add` operation for `Int`.

Recognized operator mappings:

Operator	Operation
`+`	`Add::add`
`-`	`Sub::sub`
`*`	`Mul::mul`
`/`	`Div::div`
`%`	`Rem::rem`
unary `-`	`Neg::neg`
`==`	`Equal::equal`
`!=`	`Equal::not_equal`
`<`	`Compare::less`
`<=`	`Compare::less_eq`
`>`	`Compare::greater`
`>=`	`Compare::greater_eq`
&&	`And::and` with a thunked right operand
`||`	`Or::or` with a thunked right operand
`!`	`Not::not`

Table 3: Recognized operator mappings

&& and `||` are short-circuit boolean operators. They are recognized mappings to `And::and` and `Or::or`, but the right operand is passed as a zero-argument function instead of being evaluated before the operation call.

`a && b` desugars to a call equivalent to `and(a, fn() { b })` after resolving an available `And::and` operation. `a || b` follows the same rule with `Or::or`. Both operators are defined only for `Bool`.

Concrete expression precedence, associativity, and unary/binary disambiguation follow the MoonBit parser reference where Lane2 has not deliberately removed a feature.

12 Evaluation

Lane2 v1 is pure and strict.

- Evaluating a safe expression depends only on its inputs and produces a value.
- Function arguments are evaluated before the function body runs.
- `if` and `match` evaluate only the selected branch.
- There is no mutable binding, mutable field, assignment, field update, or implicit statement sequencing.
- IO and other effects are not part of v1 core semantics.

The `Unit` value is written explicitly as `()`.

Empty blocks are invalid. A block must contain exactly one final expression after any local items.

Function calls evaluate all arguments before entering the function body. The only v1 operator forms that delay a subexpression are && and `||`, which pass the right operand as a thunk to the resolved prelude operation.

13 Undefined Behavior

Incorrect builtin use can produce undefined behavior. Lane2's type safety guarantee applies only to programs that do not misuse `builtin`.

Invalid integer arithmetic is undefined behavior in v1. This includes signed 64-bit overflow, division by zero, remainder by zero, `MIN_INT / -1`, and negating `MIN_INT`.

14 Conformance

The words "must", "must not", "may", and "should" are normative in this document.

Text marked as rationale, examples, or notes is informative unless it explicitly uses normative language.

An implementation conforms to this draft if it accepts all valid programs described here, rejects invalid programs described here, and gives the specified typing and evaluation behavior for safe programs.

Programs that misuse `builtin` are outside Lane2's safety guarantee.

15 Appendix

15.1 Deliberately Omitted From V1

V1 omits:

- mutation and assignment,
- field update and copy update,
- modules and imports,
- local type definitions,
- labeled function parameters,
- labeled enum payloads,
- type aliases,
- traits, typeclasses, interfaces,
- method syntax,
- tuple types,
- collections,
- pattern guards,
- or-patterns,
- as-patterns,
- `is` pattern expressions,
- placeholder pipeline.